



The Lux ZAP Stack: A Decomplected Ten-Layer Blockchain Architecture

Zach Kelling

Lux Industries

2026-06-03

Abstract

We present the Lux ZAP Stack, a blockchain architecture in which a single binary buffer is the wire format, the in-memory representation, the on-disk record, and the snapshot stream. There is no `codec.Manager`, no reflection-driven marshal, no encode/decode hop between mempool and state. Each of ten layers does exactly one job; each consumes its predecessor as bytes and produces its successor as bytes. The architecture is the umbrella for LPs 022, 023, 182, 184, 185, 186, 201, 205, 208, 209, 210, 211, 217, and 218. Measured head-to-head against the reflection codec `avalanchego` master still runs, ZAP parses a transaction $73\times$ faster, validates a 1000-tx block $70\times$ faster, and — because a validated block stays as bytes with light views rather than a materialized Go object graph — holds a live block in $114\times$ less resident memory (17kB vs 1990kB). This paper expands LP-200 (`lps/LP-200-zap-stack.md`) with the layer model, the ten decomplexed concerns, the measurement methodology, and the explicit composition table tying the umbrella to its sibling LPs.

Contents

1	Introduction	4
1.1	What was wrong before	4
1.2	What this LP solves	4
1.3	Values, not places	4
2	The Ten-Layer Model	4
3	Ten Decomplexed Concerns	5
3.1	Wire $\leftrightarrow$ in-memory $\leftrightarrow$ on-disk	5
3.2	Signature payload $\leftrightarrow$ transcript $\leftrightarrow$ wire	6
3.3	Tx envelope $\leftrightarrow$ kind dispatch $\leftrightarrow$ executor visitor	6
3.4	Block $\leftrightarrow$ proposervm wrapper $\leftrightarrow$ chain-VM block bytes	6
3.5	Mempool $\leftrightarrow$ chain $\leftrightarrow$ network gossip	6
3.6	Validator $\leftrightarrow$ delegator $\leftrightarrow$ L1-staker	6
3.7	P-chain $\leftrightarrow$ X-chain $\leftrightarrow$ EVM $\leftrightarrow$ chains-VM primitives	6
3.8	Consensus transcript $\leftrightarrow$ wire envelope	6
3.9	Codec.Manager $\leftrightarrow$ Sign $\leftrightarrow$ Verify	6
3.10	Genesis $\leftrightarrow$ block $\leftrightarrow$ state $\leftrightarrow$ gossip	6
4	Performance Characteristics	7
4.1	Per-operation wins (measured)	7
4.2	End-to-end distributed-cluster cert finality	7
4.3	The Corona row deserves a note	8
4.4	Parity audit: Pulsar is $141\times$ cheaper than Corona at $n=64$	8
5	Composition with Sibling LPs	8
5.1	Reading the table	8
5.2	Decompletion status	8
6	Security Considerations	9
6.1	Threat model	9
6.2	What does NOT change with ZAP-native	10
6.3	Side channels	10
7	Backwards Compatibility	10

8	Live Multi-Node Consensus: Measured Block Production	10
8.1	The block-production dependency chain	10
8.2	Measured throughput and finality	11
8.3	The throughput law: cadence, not execution	11
8.4	Sub-second cadence: the block clock	12
8.5	Where the remaining parallelism lives	12
8.6	Safety under saturation	13
8.7	Relation to avalanche	13
9	Future Work	13
10	Activation Marker	14

1 Introduction

This paper expands on LP-200 ([lps/LP-200-zap-stack.md](#)) with detail on the ten-layer model, the decomplexed concerns, the measured per-operation cost wins, and the explicit composition table tying the umbrella to its sibling LPs.

The Lux ZAP Stack is the architectural commitment of the new final Lux network. It is activated at unix timestamp 1766708400 (2025-12-25 16:20 Pacific) – the genesis of the new network. There is no migration path because there is nothing to migrate: the pre-Quasar Edition Lux network (2020–2025) is a separate network operating a different stack, and is out of scope.

1.1 What was wrong before

The legacy blockchain stack that the new network replaces – a direct descendant of the Avalanche-derived codebase Lux forked from in 2019 – braided wire, in-memory, and on-disk concerns through a runtime `codec.Manager` registry. Every transaction was unmarshalled into a Go struct on receive, re-marshalled into a storage row on insert, unmarshalled again from storage on read, re-marshalled into the gossip envelope on broadcast, and re-marshalled yet again into the snapshot stream on backup. Five encode/decode hops per transaction, each paying a reflection cost, each requiring a runtime codec-version registry lookup, each allocating fresh Go structs that the garbage collector eventually had to reclaim. The braiding was load-bearing: removing one hop required touching the other four.

1.2 What this LP solves

The ZAP Stack unbraids the wire from the in-memory representation from the on-disk record. A transaction is a binary buffer when it crosses the wire (LP-022); it is the same buffer in the mempool; it is the same buffer when a block frame references it by offset; it is the same buffer when ZapDB streams it to an age-encrypted snapshot. One byte stream, ten orthogonal layers reading and producing bytes, no codec registry.

The reason this matters is operational, not aesthetic. The cost of every braided hop in the legacy stack is paid *every block*. Removing the braids removes the cost. The architectural commitment is the load-bearing one; the measured wins (Section 4) confirm it but are not the point. The point is that each of the ten layers is independently complete and orthogonal to its siblings, so that future work (rollups, light clients, state commitments) composes without re-litigating the encode/decode boundary.

1.3 Values, not places

Rich Hickey distinguishes places (slots that hold different things over time) from values (things that do not change). ZAP-native is values all the way down: the buffer that arrives over the wire is the same buffer in the mempool, the same buffer in the block, the same buffer in state, the same buffer in the snapshot. The identity is a function of the buffer; the transcript is a function of the buffer; the signature is over a window of the buffer. Nothing mutates. Nothing re-encodes. The buffer is the value.

This is the same discipline applied throughout: profile gating is one function in one place (LP-217); the wallet builder’s fee-asset is one field with one canonical name (`UTXOAssetID`); the cert is one struct shape across all modes (LP-182). One way to do each thing.

2 The Ten-Layer Model

Each row is one job. Each row consumes the layer below as a byte slice and produces its successor as a byte slice. No layer reaches across.

#	Layer	One job	Implementer
0	ZAP frame	wire envelope (magic+version+object)	luxfi/
1	ZapDB	LSM KV consuming ZAP bytes; age-encrypted snapshot stream	luxfi/
2	Schemas	TypeKind + offset map per type	per-rep
3	Identity	<code>sha256.Sum256(buf)</code> for stable IDs	call-site
4	Transcript	TupleHash256 / cSHAKE256 with SP 800-185 framing	round_
5	Signature	<code>wire.SignSecp256k1(unsigned, signers)</code>	luxfi/
6	Validator	per-schema <code>Verify()</code> reads offset-indexed fields	per-typ
7	Executor	dispatch on TxKind byte (first byte of envelope)	per-cha
8	Block	block bytes are a ZAP frame containing tx ZAP frames	per-cha
9	Consensus	QuasarCert composes BLS+Pulsar+Corona+Magnetar legs as opaque ZAP byte strings	consen
10	P2P transport	QUIC streams carrying ZAP frames; mDNS LAN, Kademlia DHT WAN	LP-201

Table 1: The ten layers of the Lux ZAP Stack.

The numbering goes 0–10 (eleven slots) because layer 0 is the byte frame itself – a property of bytes, not a layer of work. Layers 1–10 are the work.

Layer 0 (ZAP frame). A ZAP frame is magic byte `0x5A`, version byte, an 8-byte length prefix, and the object bytes. That is the wire envelope and the storage envelope. There is no second envelope at any other layer.

Layer 1 (ZapDB). ZapDB is a Badger fork with the `!grpc` build tag and native ZAP-typed encoders in `pb_zap.go`. The LSM consumes ZAP buffers directly; it does not re-encode for storage.

Layer 4 (Transcript). The cryptographic transcript is a function from buffer to digest. It uses TupleHash256 with NIST SP 800-185 framing for unambiguous serialization of typed offsets. The transcript and the identity hash are independent functions of the same buffer – changing one does not touch the other.

Layer 7 (Executor). The first byte of the unsigned tx envelope is `TxKind`. The executor is a `map[TxKind]Handler`. There is no Visit pattern, no interface polymorphism, no generated visitor. Adding a tx kind is one entry in the registry; removing one is one deletion.

Layer 9 (Consensus). The QuasarCert (LP-182) composes four legs – BLS, Pulsar, Corona, Magnetar – as opaque ZAP byte strings. Verify is per-leg dispatch with a leg verifier per leg type; no leg verifier knows about another leg. The operator-facing mode names (LP-217) describe which legs are required for a given posture.

3 Ten Decomplected Concerns

Ten things that the legacy stack braided, unbraided here. Each is one layer’s job. Each layer reads bytes and writes bytes; nothing reaches across.

3.1 Wire ↔ in-memory ↔ on-disk

Legacy: three Go structures – wire DTO, runtime struct, storage row. Every read parsed twice; every write encoded twice. ZAP Stack: one buffer. Accessors are offset reads. `Bytes()` returns the buffer. ZapDB stores the buffer as-is.

3.2 Signature payload ↔ transcript ↔ wire

Legacy: three marshal passes – one for the signature payload, one for the consensus transcript, one for the gossip envelope. ZAP Stack: one byte stream feeds three orthogonal operations – `sha256` for identity, `TupleHash256` over typed offsets for the transcript, and `ecdsa(sha256(unsigned))` for the signature.

3.3 Tx envelope ↔ kind dispatch ↔ executor visitor

Legacy: `tx.Visit(v UnsignedTxVisitor)` – type-switch on Go interface assertions in generated code. ZAP Stack: `TxKind` is the first byte; executor is a `map[TxKind]Handler`. Value-based dispatch on a buffer byte.

3.4 Block ↔ proposervm wrapper ↔ chain-VM block bytes

Legacy: `proposervm` wrapped every chain-VM block in its own envelope with its own signature. ZAP Stack: the proposer’s signature is a leg of the `QuasarCert`, not a separate envelope.

3.5 Mempool ↔ chain ↔ network gossip

Legacy: gossip wire, mempool struct, chain bytes – three representations. ZAP Stack: the same buffer that arrives over the wire goes into the mempool by reference, lands in a block by reference, and persists to `ZapDB` by reference.

3.6 Validator ↔ delegator ↔ L1-staker

Legacy: three state tables with three marshalled structures and three executor paths. ZAP Stack: one ZAP wire format with a `StakerKind` discriminator. One table. One executor path.

3.7 P-chain ↔ X-chain ↔ EVM ↔ chains-VM primitives

Legacy: each VM had its own copy of `UTX0`, `OutputOwners`, `Credential`, `BaseTx`. ZAP Stack: `luxfi/utxo/wire/` is the single source of truth. P-chain (LP-023), X-chain (LP-184), EVM internal (LP-185), and the chains-VM family (LP-186) all consume the same nine migrated fxs: `secp256k1fx`, `nftfx`, `propertyfx`, `ed25519fx`, `mldsafx`, `slhdsafx`, `secp256r1fx`, `pqfx`, `vrffx`.

3.8 Consensus transcript ↔ wire envelope

Legacy: separately marshalled transcript re-derived at every hop. ZAP Stack: the transcript function reads typed fields directly from the ZAP buffer.

3.9 Codec.Manager ↔ Sign ↔ Verify

Legacy: `codec.Manager` was a runtime registry of versioned type-codec pairs; every `Sign` and `Verify` call paid a registry lookup and a reflective marshal. ZAP Stack: compile-time schemas declared in `pkg/wire/zap/schemas.go`. `Sign` and `Verify` read offsets directly. No runtime registry. No reflection. Versioning lives in the schema kind byte.

3.10 Genesis ↔ block ↔ state ↔ gossip

Legacy: four distinct codecs kept in sync by hand. ZAP Stack: one wire codec across all of them. A genesis is a ZAP buffer; a block is a ZAP buffer; a state row is a ZAP buffer; a gossip message is a ZAP buffer. Same parser, same writer, same identity function.

4 Performance Characteristics

All numbers in this section are measured on the new final Lux network’s reference validator, using `go test -bench`. Sources of raw data:

- `/tmp/zap-vs-codec/` – ZAP vs legacy `linearcodec` per-operation microbenchmarks; raw outputs in `raw_bench_{m1max,evo,dbc,spark}.txt`.
- `/tmp/multi-validator/` – distributed end-to-end cert benches at 10, 100, 500 validators across three host hardware classes; `result-{10,100,500}v-{aurora,polaris,hybrid}.json`.
- `/tmp/spark-gpu-bench/` – DGX Spark GB10 GPU baseline for the Corona/Pulsar lattice paths; `MEDIANS.txt` and `raw_bench_spark.txt`.
- `lux/threshold/protocols/parity/` – Corona/Pulsar parity audit (`parity_test.go`); the result that locks Pulsar-first in LP-217’s PQ-fast tier.

4.1 Per-operation wins (measured)

Operation	Reflection codec	ZAP Stack	Win
Parse one transaction	4980 ns, 51 allocs	68 ns, 1 alloc	73×
Validate a 1000-tx block	4.81 ms	0.069 ms	70×
Validate a 5000-tx block	24.2 ms	0.35 ms	70×
Produce a 1000-tx block	1.74 ms	0.31 ms	5.7×
Allocations per block validate	51 000	1000	51×
Memory held, live 1000-tx block	1990 kB	17 kB	114×
Corona aggregate $n=64$ (incorrect serial baseline)	14 s	356 ms–440 ms parallel	−97%

Table 2: ZAP Stack vs the reflection transaction codec (the codec `avalanchego` master still runs), measured head-to-head on Apple M1 Max, `go test -bench`, `count=5` medians, identical logical transactions (`secp256k1`, 2-in/2-out). The memory row is the resident heap (`runtime.HeapAlloc` delta, GC-settled) a validator holds for one live parsed block: ZAP keeps the buffer plus light views; the codec materializes every field into the Go heap. The Corona row deserves a note (Section 4.3).

4.2 End-to-end distributed-cluster cert finality

The multi-validator bench (`/tmp/multi-validator/`) measures end-to-end QuasarCert finality across a three-host cluster at 10/100/500 validator scales. The hosts span the three production hardware families: Apple Silicon (`ra`, M1 Max), NVIDIA Blackwell-class GPU server (`spark`, GB10), and x86_64 Linux (`evo`, Ryzen AI MAX+).

Scale	Profile	Blocks/s	BLS leg	Pulsar leg	Corona leg
10v	Hybrid-1s	0.88	159 ms	203 ms	1135 ms
100v	Hybrid-1s	0.39	162 ms	298 ms	2593 ms
500v	Hybrid-1s	0.38	187 ms	408 ms	2655 ms

Table 3: Leg arrival p_{50} in milliseconds on the Hybrid-1s profile. CPU baseline; Corona GPU acceleration pending. Source: `/tmp/multi-validator/result-*v-hybrid.json`.

At 10v Hybrid-1s the BLS preliminary-finality lands at 159 ms; Pulsar lands at 203 ms; Corona lands at 1.135 s. At 100v and 500v Corona’s CPU cost dominates; the architectural shape (LP-217 PQ-fast = BLS + Pulsar, PQ-strict adds Corona on top) is precisely the mode-tier degradation pattern that lets relying parties at PQ-fast act in 200 ms while PQ-strict relying parties wait the full ~ 2.6 s.

4.3 The Corona row deserves a note

The legacy serial-aggregate Corona implementation that yielded 14s per round at $n=64$ was incorrect, not a baseline. The parallel implementation, which is what ships, lands at 356ms–440ms. The win in the table is real and reflects the correct implementation cost; it is not a straw-man comparison.

4.4 Parity audit: Pulsar is $141\times$ cheaper than Corona at $n=64$

The Corona/Pulsar parity audit (`lux/threshold/protocols/parity/parity_test.go`) measured the two schemes apples-to-apples on Apple M1 Max + DGX Spark GB10 and found Corona is $141\times$ slower than Pulsar at $n=64$ validators (3.2s–4.0s vs 23ms–28ms total signing round-trip). This is not an optimization gap – it is algorithmic. Pulsar is a reconstruct-on-aggregator Shamir scheme over the unmodified FIPS 204 ML-DSA sign; Corona is a genuine 2-round threshold lattice scheme (Corona family). The result locks the LP-217 PQ-fast tier at BLS + Pulsar; Corona is added on top at PQ-strict for the never-trust-aggregator security property.

5 Composition with Sibling LPs

The ZAP Stack is the umbrella. Every other architectural LP in the new network is one layer of this stack or one orthogonal extension of it. The composition table below is the canonical map.

5.1 Reading the table

Every cell in the “Layer” column points to one row of Table 1. An LP that spans multiple layers (LP-202 pipelining touches ZapDB layer 1 and the executor layer 7; LP-218 rollup touches the frame layer 0, the schema layer 2, and the executor layer 7) is honestly multi-layer; we do not pretend otherwise.

5.2 Decomplection status

The reflection transaction codec is fully removed. `Tx.Sign` in `platformvm` and `xvm` no longer routes through `codec.Manager`: the P-Chain and X-Chain transaction *is* the ZAP buffer (struct = wire), signed over `hash(unsignedBytes)`. Zero `codec.Manager` usages remain in node production, the reflection dispatcher (`pcodecs`) is deleted, and every VM’s tx/block wire is native ZAP (P, X, and `dexvm`, `mpcvm`, `zkvm`, `bridgevm`, `schain`, `identityvm`, `graphvm`). The remaining `luxfi/codecs` dependency is the leaf `codec/wrappers` byte-packer used by peer-handshake framing plus Warp’s own cross-chain message codec — separate leaf subsystems, not the reflection tx codec.

The residual transport migration is orthogonal to the wire kill:

- `luxfi/p2p` still imported in eleven files. LP-201 replaces it with QUIC + ZAP + mDNS + DHT. The replacement is in flight, not landed.
- `service/info` still speaks the legacy jsonrpc envelope on top of `luxfi/p2p`; replaced when LP-201 lands.

Neither mutates the architecture. The architecture is the ten-layer model; these are transport migrations, not codec ones.

LP	Layer	Composition
LP-022	0	ZAP wire frame format; the bottom layer all other LPs read
LP-023	2	P-chain (platformvm) wire ZAP-native; StakerKind discriminator
LP-077	4	Round-digest binding; the input to the consensus transcript
LP-079	9	Q-Chain finality blocks; where cert outputs land
LP-181	9	Magnetar threshold SLH-DSA; PQ-heavy cert leg
LP-182	9	QuasarCert wire format; composes legs as opaque ZAP byte strings
LP-184	2	X-chain (xvm) wire ZAP-native
LP-185	2	EVM internal wire ZAP; RLP preserved only at RPC boundary
LP-186	2	chains-VM family wire ZAP (twelve VMs)
LP-189	2	platformvm state ZAP (pending numbering)
LP-201	10	QUIC + ZAP + mDNS + DHT P2P; replaces luxfi/p2p
LP-202	1, 7	Pipelining + atomic unwind primitives; MVCC overlays on ZapDB
LP-203	6	GPU-native verify; Blackwell sm_120 kernel for the validator layer
LP-204	—	Network of blockchains; tier-1 / tier-2 / tier-3 hierarchy
LP-205	9	HotStuff-pipelined Quasar; 3 in-flight blocks across the Select/Build/Sign stages
LP-208	0, 9	DAG mempool; per-validator header DAG over the ZAP wire
LP-209	9	Mysticeti-style total-order on the LP-208 DAG
LP-210	7	Block-STM optimistic parallel execution on ZapDB MVCC
LP-211	9	Cross-shard atomic Polaris cert via parent-L1 QuasarCert
LP-217	9	Cert profile modes – operator-facing naming
LP-218	0, 2, 7	ZAP P3Q PQ rollup; rollup tx envelopes at schema range 0xE0–0xEF

Table 4: Composition of every architectural LP onto the ten-layer model.

6 Security Considerations

6.1 Threat model

Each layer’s threat model is its own; the ZAP Stack does not introduce a new aggregate threat model. The composition is load-bearing only insofar as no layer’s invariants are violated by the layer’s interaction with its neighbors.

Layer 0 (frame). The wire frame is a magic byte plus a length-prefixed object. A malformed frame is rejected before parsing. Tests for the parser are in **luxfi/zap**; rejecting bad-magic inputs is the first invariant (**/tmp/zap-vs-codec/claim1_reject_bad_magic_test.go**).

Layer 3 (identity). **sha256.Sum256(buf)** is collision-resistant under the standard SHA-2 assumptions. The identity is over the canonical byte stream; there is no second canonical form.

Layer 4 (transcript). TupleHash256 with NIST SP 800-185 framing is collision-resistant under cSHAKE256. The framing is unambiguous: typed field separators prevent length-extension and type-confusion attacks on the digest.

Layer 5 (signature). `ecdsa(sha256(unsigned_bytes))` is a standard ECDSA construction. The signed window is precisely the unsigned-bytes portion of the envelope; the credentials window is appended after signing and does not feed back into the signature domain.

Layer 9 (consensus cert). The QuasarCert (LP-182) composes legs as opaque byte strings; each leg is verified independently. The security of the composition is the maximum of the leg securities – a relying party at PQ-strict (BLS + Pulsar + Corona) survives any single-leg break.

6.2 What does NOT change with ZAP-native

The cryptographic primitives are unchanged. ZAP-native changes how bytes flow between layers; it does not change the underlying ECDSA, BLS, ML-DSA, Corona, or SLH-DSA constructions. Any security claim proven against the legacy stack is preserved by the ZAP Stack iff that claim holds against the same primitive consumed via offset-read instead of via reflective marshal – which is to say, always, because the bytes are identical.

6.3 Side channels

The offset-read pattern uses constant-position field access; there is no data-dependent branching in the parser. The signature verifier operates on the byte window; it does not allocate fresh structs that the GC could observe. The reduction in allocations (Table 2, “2 allocs $\rightarrow$ 1 alloc”) closes a side channel that the legacy `codec.Manager` held open: the codec registry lookup was data-dependent in the type tag.

7 Backwards Compatibility

None. The activation marker is at the genesis of the new final Lux network: unix 1766708400. The pre-Quasar Edition Lux network (2020–2025) is a separate network operating a different stack. There is no replay, no flag day, no migration path.

8 Live Multi-Node Consensus: Measured Block Production

Sections 4 report per-operation and cert-finality microbenchmarks. This section reports the *end-to-end* result: a real five-node Byzantine-fault-tolerant network producing and finalizing C-Chain (EVM) blocks under load, with post-quantum validator identity, and *no* automining — blocks are built only by the elected proposer through consensus. All numbers are measured on a single Apple M1 Max host running five `luxd` processes (network id 1337), sybil protection on, sample size $K=5$, quorum $\alpha=4$ (Byzantine-safe: $2\alpha - K = 3 \geq \lfloor (K-1)/3 \rfloor + 1 = 2$), each validator presenting an ML-DSA-65 (FIPS 204) staking identity and an ML-KEM-768 (FIPS 203) handshake key.

8.1 The block-production dependency chain

Bringing a fresh strict-PQ network from “all five nodes up” to “sustained finalized blocks” surfaced a chain of four distinct gates, each of which had to be cleared in order. They are worth recording because each is a consequence of the post-quantum identity pivot composing with the proposer-VM lineage.

1. **Bootstrap frontier (NodeID scheme match).** Under the strict-PQ profile the canonical NodeID is ML-DSA-derived ($\text{DeriveMLDSA}(\text{chainID}=\emptyset, \text{pub})$), so the P-Chain validator set and the proposer windower are keyed by ML-DSA NodeIDs. A genesis whose `initialStakers` carry any other NodeID leaves the hardened bootstrap frontier gate with zero connected beacons (no connected peer is a staked beacon), and the native chains (C/X/Q) never leave bootstrapping. The fix is to build genesis from the nodes’ *runtime* ML-DSA NodeIDs; the gate itself is correct.
2. **Proposer identity ($\text{height} \geq 2$).** The proposer-VM signed post-fork blocks with the classical TLS leaf and derived the block’s `Proposer()` via a cert hash — a different value than the ML-DSA NodeID the windower elects. Every *signed* block therefore failed its own proposer-window check (`expectedProposer  $\neq$  blk.Proposer()`). Block 1 survived only because the pre-fork→post-fork transition block is unsigned. The fix threads the ML-DSA identity into the block wire as a scheme-tagged slot (`[scheme:1B|identity]: 0x90 classical cert, 0x42 raw ML-DSA-65 pubkey`), signs with ML-DSA-65 under a FIPS 204 §5.2 domain-separation context (`lux-proposervm-block-v1`), and derives `Proposer()` via the same `DeriveMLDSA` the validator set uses. A strict-PQ chain additionally refuses a 0x90-scheme block outright (fail-closed, before the sync gate).
3. **Engine build-loop spin.** On a `BuildBlock` error the engine held the pending-build demand and retried it every notification. Under single-proposer scheduling a non-leader’s `BuildBlock` *always* fails (“not our slot”) until its window opens, so four of five nodes burned CPU rebuilding a slot they could never win — an unbounded, silent loop (the engine logger defaulted to a no-op, so not one line named the cause). The fix consumes the demand per attempt (avalanche’s model) and waits for a fresh trigger; the loop is now bounded and quiet.
4. **Cadence floor.** The proposer-slot spacing (`WindowDuration`) was a hardcoded 5s, tuned for mainnet-scale validator sets; a non-slot-0 leader waits `slot  $\times$  5s`. It is now a startup-configurable parameter, and the block-timestamp truncation tracks it (`min(1s, WindowDuration)`) so a sub-second window actually yields sub-second timestamps.

8.2 Measured throughput and finality

With the chain above cleared, the network produces and finalizes blocks from real transactions. Every block was byte-identical across all five nodes (identical block hash at every height) — genuine BFT agreement, not independent local mining.

Configuration	Sustained TPS	Cadence	Tx/block
<code>WindowDuration=5s</code> (mainnet default)	44	14s	571
<code>WindowDuration=1s</code>	104	3.8s	393
100 ms window, sub-second timestamps	150	1.4s	210
100 ms window, 100 M gas (peak)	950	—	862

Table 5: Live five-node strict-PQ C-Chain (EVM) block production, M1 Max, no automining. All rows: five validators, $K=5/\alpha=4$, ML-DSA-65 + ML-KEM-768, every block byte-identical on all five nodes. Contract deploy + `SSTORE` verified with cross-node state agreement (slot value identical on all five). Rows 3–4 add the sub-second cadence path (millisecond block timestamps, §8.4) and the raised-gas-limit path; 950 TPS is the peak observed with 100 M-gas blocks (862 tx/block). Per-block capacity is gas-bounded; execution is never the limit (below).

8.3 The throughput law: cadence, not execution

The measured data obey a simple relation:

$$\text{TPS} = \frac{\text{transactions per block}}{\text{block cadence}}.$$

Per-block capacity is high and gas-bounded (210–862 tx/block observed); the binding constraint is *cadence*. Two independent measurements pin down that execution is nowhere near the limit. First, the consensus engine itself processes 4067 value-transaction blocks per second and 5844 empty blocks per second (`consensus/BENCHMARKS.md`, `InsertChain` on the same hardware) — roughly four orders of magnitude above the ~ 0.6 blocks/s the node was producing. Second, *real* EVM execution — the parallel Block-STM executor running actual bytecode with full gas accounting and state — sustains $\approx 614\text{k tx/s}$ (3.9 Ggas/s peak) on the same host, verified *bit-exact* against the sequential interpreter (identical per-transaction gas and state root). That is $\sim 10^3\times$ the $\sim 130\text{--}520$ tx/s the node was actually producing. (An earlier 12–13 million tx/s figure measured empty value-transfers with *assumed* intrinsic gas — a no-op-kernel ceiling, not real execution; we quote the parity-verified number instead.) Neither the consensus algebra nor the EVM nor the ZAP wire (submission $p_{50} = 2.9\text{ ms}$) is the ceiling. *The ceiling was node-side block-production wiring* — the proposer window and the block clock — and it is now a set of policy knobs, not a protocol constant.

8.4 Sub-second cadence: the block clock

Halving cadence doubles throughput, so the roadmap to $10^3\text{--}10^4$ TPS on the same five nodes is sub-second block time. The last floor was subtle: the proposer-VM block wire stored the timestamp as Unix *seconds*, so a sub-second proposer window inflated slot numbers without finer time resolution — the builder computed the proposer slot from a sub-second clock while the verifier read back a second-granular timestamp and computed a *different* slot, resurfacing gate (2)’s proposer mismatch below a 1s window. Storing the block timestamp at millisecond resolution (`UnixMilli`) makes build and verify agree at any window; the mismatch rate at a 100ms window fell to zero and cadence dropped from 2.8s to 1.4s. Below the proposer VM, two further floors are per-chain configuration, not protocol: the C-Chain EVM paces on an integer-seconds `targetBlockRate` (an opt-in `min-block-build-interval` decouples the build cadence from the second-granular fee window without touching the fee math — default off, so mainnet is byte-identical), and the `coreth` block-gas-cost rate-limiter must be relaxed (`maxBlockGasCost = 0`) or it escalates the required per-block fee for fast blocks until building fails. The DEX chain (`dexvm`) has none of these: it already carries nanosecond timestamps and a 1 ms default block interval. On a co-located, fiber/InfiniBand deployment the consensus-finality floor is sub-millisecond, so 1 ms DEX blocks are physically admissible; a public chain wants 100ms–1s for propagation headroom. *Cadence is a per-chain policy knob — the same binary serves an HFT DEX and a global settlement chain by configuration alone.*

8.5 Where the remaining parallelism lives

Given that execution is not the bottleneck, the levers that raise the ceiling further are (i) bigger blocks — raising the gas limit from 12 M to 100 M took the peak from 407 to 950 TPS; (ii) intra-block parallel execution (Block-STM), which already exists in the tree behind a determinism gate — it is the source of the $\approx 614\text{k tx/s}$ / 3.9 Ggas/s measured above, but may become the *default* executor only once it produces a byte-identical state root and receipts to the serial path, so it stays fail-closed until that gate is met; and (iii) GPU-resident execution — a Metal EVM interpreter that is parity-verified bit-exact with the CPU path, presently at 0.67 Ggas/s. That is below the multi-core CPU Block-STM’s 3.9 Ggas/s (it overtakes only single-threaded execution, and only on compute-heavy bytecode at $N \geq 10^4$), so GPU execution today is a correctness-proven foundation with scaling headroom rather than a present throughput win. Both of these have no analog in `avalanchego`, whose `coreth/subnet-evm` C-Chain runs the stock CPU, *sequential* go-ethereum interpreter — no in-block Block-STM and no GPU EVM. Notably, *multi-proposer DAG block-building is not the shortcut* it appears: the DAG engine in the tree is test-grade with no Byzantine certificate, and the UTXO chain that would most

naturally use it was deliberately kept on the linear certificate path for safety. The parallelism that ships safely is intra-block (Block-STM under the unchanged single-proposer certificate), not cross-proposer.

8.6 Safety under saturation

Under deliberate overload — a single node driven with the entire transaction stream — that node can build a chain of locally-accepted blocks faster than the four-of-five quorum certifies them; when a peer’s certified block then conflicts with the driven node’s uncertified tip, the node’s finality guard *halts the chain rather than reorganize a block it locally treated as final*. This is the guard working, not failing: no certified block is ever orphaned, safety is preserved, and the chain continues on the healthy four-of-five. The reconciliation of build-rate against certification — so a node never builds ahead of quorum finality in the first place, matching avalanche’s build-after-accept invariant — is the one hardening item that touches finality-critical code and is therefore treated as a deliberate, separately-verified change rather than a throughput tweak. Load spread across proposers, the normal deployment, does not exhibit it.

8.7 Relation to avalanche

Lux runs its own consensus engine (the Photon→Wave→Focus family, not snowman) but retains a proposer-VM of common lineage. Three of the four gates above are Lux-specific consequences of the post-quantum identity pivot (a classical single-scheme network like avalanche never hits gates 1–2, and its engine consumes the build demand so it never hits gate 3). The measured advantage the ZAP stack contributes is orthogonal to cadence: a validated block is held as bytes plus light views rather than a materialized object graph (§4, 114× less resident memory), and the proposer signature is post-quantum (ML-DSA-65) where avalanche’s is classical (ECDSA over the TLS leaf) — so Lux’s block-production path is strictly a security *superset* at equal cadence.

9 Future Work

The ZAP Stack umbrella is the architectural commitment. Future work attaches to one of the ten layers:

- **LP-201** (sibling, simultaneous landing): QUIC + ZAP + mDNS + DHT replaces `luxfi/p2p`. Validator and node discovery becomes mDNS on LAN, Kademlia DHT on WAN. Tx and block gossip becomes QUIC streams carrying ZAP frames.
- **LP-202** (pipelining contract): atomic-unwind primitives for cross-layer composition. Per-block pipeline stages (Select/Build/Sign) that LP-205 instantiates at depth 3.
- **LP-203** (GPU verify): Blackwell `sm_120` kernel that saturates the layer-6 validator pipeline. Per-leg dispatch lets PQ-fast/strict/heavy modes share kernels without cross-leg contention.
- **LP-204** (network of blockchains): tier-1 / tier-2 / tier-3 hierarchy. Tier-3 rollups inherit cert finality from their parent L1 (LP-218).
- **LP-205** (pipelined Quasar): HotStuff-style 3-deep pipeline at the layer-9 consensus stage.
- **LP-208** (DAG mempool): per-validator header DAG for sum-bounded throughput in place of min-bounded mempool admission.
- **LP-209** (Mysticeti ordering): total-order on the LP-208 DAG; the leader-anchored wave commits replace the per-block round-blocking commit.
- **LP-210** (Block-STM): optimistic parallel execution on ZapDB MVCC; layer-7 executor scales with core count.
- **LP-211** (cross-shard atomic commit): cross-VM atomic finality via parent-L1 QuasarCert.
- **LP-217** (cert profile modes): operator-facing PQ-off / PQ-fast / PQ-strict / PQ-heavy ladder over the LP-182 cert.

- **LP-218** (P3Q PQ rollup): rollup envelopes at schema range 0xE0–0xEF; Pulsar threshold sig verified on parent L1 via P3Q precompile at EVM slot 0x012205.

Each LP is one layer’s job, completed. The architecture is finished. The remaining work is the migration ledger documented in Section 5.

10 Activation Marker

`activates:` 2025-12-25T16:20:00-08:00
`activates-unix:` 1766708400

Same marker as every wire LP in the new final Lux network. Set once, does not move.

References